

مرحله ۱: تعریف View در SQL برای فرم جدید

برای هر فرم جدید، ابتدا باید یک View در SQL ایجاد کنید که داده‌های فرم را آماده نمایش کند.

نکات مهم:

نام View:

همیشه باید با Vw شروع شود و با Form خاتمه یابد.

مثال: اگر فرم مربوط به ComponentType باشد، نام View باید باشد:

VwComponentTypeForm

جدول اصلی: جدول مرتبط با فرم (مثلاً ComponentTypes).

فیلدهای ضروری که همیشه باید باشند:

CreatedAtPersian → تاریخ ایجاد به تقویم شمسی

UpdatedAtPersian → تاریخ آخرین بروزرسانی به تقویم شمسی

CreatedByName → نام کامل کاربری که رکورد را ایجاد کرده

UpdatedByName → نام کامل کاربری که رکورد را آخرین بار بروزرسانی کرده

نمونه SQL:

```
ALTER VIEW [cms].[VwComponentTypeForm]
```

```
AS
```

```
SELECT
```

```
ct.[Id]
```

```
ct.[Name],
```

```
ct.[Code],
```

```
ct.[Type],
```

```
ct.[Description],
```

```
ct.[IsActive],
```

```
dbo.GregorianToPersianDateTime([CreatedAt]) AS CreatedAtPersian,
```

```
dbo.GregorianToPersianDateTime([UpdatedAt]) AS UpdatedAtPersian,
```

```
cu.FullName AS CreatedByName,  
uu.FullName AS UpdatedByName,  
FROM [CoreCms].[cms].[ComponentTypes] ct  
LEFT JOIN [auth].[VwUsersLite] cu ON cu.Id = ct.CreatedBy  
LEFT JOIN [auth].[VwUsersLite] uu ON uu.Id = ct.UpdatedBy
```

⚠ نکته مهم:

این چهار فیلد (CreatedAtPersian, UpdatedAtPersian, CreatedByName, UpdatedByName) همیشه باید وجود داشته باشند تا فرم در پنل ادمین و رابط کاربری به درستی نمایش داده شود.
نام View حتماً با Vw شروع و با Form خاتمه یابد.

مرحله ۲: ساخت مدل‌ها با Scaffolding

برای فرم جدید، بعد از تعریف View در SQL، باید مدل‌های #C فرم را از دیتابیس ایجاد کنید.

نکات مهم:

کتابخانه مورد استفاده:

از کلاس لایبرری Velora.EntityFrameworkCore استفاده کنید.

دستور اجرا:

دستورات Scaffolding آماده شده داخل فایل ScaffoldCommands.txt موجود هستند.

کافی است دستور مربوط به دیتابیس فرم مورد نظر را در کنسول اجرا کنید تا مدل‌ها ایجاد شوند.

خروجی:

مدل‌های ایجاد شده در پروژه، کلاس‌های مرتبط با جدول‌های دیتابیس و View ها را شامل می‌شوند.

این مدل‌ها باید در مسیر مناسب پروژه قرار بگیرند و در صورت نیاز نام کلاس و namespace طبق استاندارد پروژه اصلاح شود.

مرحله ۳: ایجاد مدل‌های فرم (CRUD و DTO)

برای هر فرم جدید باید دو مدل #C ایجاد کنید:

۳-۱: مدل اصلی (<EntityName>Crud)

این مدل برای فرم ثبت/ویرایش و DataGrid استفاده می‌شود.

نکته خیلی مهم: مدل CRUD باید از کلاس BulkInsert ارث ببرد.

باید همه پراپرتی‌ها با [ResourceColumn] Attribute مشخص شوند.

نکات مهم:

همه فیلدهای متنی **MaxLength** داشته باشند (مطابق **stringLength** در SQL).

فیلدهای نمایش در فرم و **Grid** با **ShowInForm** و **ShowInGrid** کنترل شوند.

فیلدهای انتخابی (**ComboBox / MultiSelectBox / SelectBox**) دو پراپرتی داشته باشند: **Id** و **Name**.

Attribute ها باید دقیقاً تنظیم شوند تا فرم، **Grid** و **ComboBox** ها درست کار کنند.

مثال:

```
public class ComponentTypeCrud : BulkInsert
{
    // حتما از BulkInsert مشتق شده باشد
```

```
    public Guid? Id { get; set; }
```

```
    ResourceColumn(FieldType = FieldTypes.Text, FormOrder = 1, GridOrder = 1, ShowInGrid = ]
```

```
        [true, ShowInForm = true, MaxLength = 100)
```

```
    ;!public string Name { get; set; } = null
```

```
    ResourceColumn(FieldType = FieldTypes.Text, FormOrder = 2, GridOrder = 2, ShowInGrid = ]
```

```
        [true, ShowInForm = true, MaxLength = 100)
```

```
    ;!public string Code { get; set; } = null
```

```
    // سایر فیلدها با Attribute مشابه...
```

```
}
```

۳-۲: مدل (<EntityName>Dto)

این مدل دقیقاً از Scaffolding تولید می‌شود.

فقط شامل پراپرتی‌ها و نوع داده‌ها است، بدون Attribute فرم و Grid.

برای انتقال داده‌ها و عملیات سرویس استفاده می‌شود.

مثال نام مدل: ComponentTypeDto

مثال ساده:

```
public class ComponentTypeDto
{
    public Guid Id { get; set; }
    public string Name { get; set; } = null;
    public string Code { get; set; } = null;
    public string Type { get; set; } = null;
    public string? Description { get; set; }
    public bool? IsActive { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
    public Guid CreatedBy { get; set; }
    public Guid UpdatedBy { get; set; }
}
```

✓ نکات کلیدی مرحله ۳:

مدل CRUD حتماً از BulkInsert مشتق شود.

همه فیلدهای متنی MaxLength داشته باشند.

فیلدهای نمایش در فرم و Grid با ShowInForm و ShowInGrid کنترل شوند.

فیلدهای انتخابی (SelectBox / MultiSelectBox / ComboBox) دو پراپرتی داشته باشند: Id و Name.

Attribute ها (ResourceColumn) باید دقیقاً تنظیم شوند تا فرم و Grid و ComboBox ها درست کار کنند.

مدل DTO فقط برای انتقال داده استفاده می‌شود و Attribute ندارد.

مرحله ۴: ایجاد فایل‌های Resource برای ترجمه (فارسی و انگلیسی)

برای هر فرم جدید، باید فایل‌های Resource مربوط به نام فیلدها و عنوان‌ها ایجاد شود تا سیستم بتواند در حالت توسعه (Development) به صورت خودکار از آن‌ها استفاده کند.

مسیر فایل‌ها 

تمام فایل‌های Resource باید داخل مسیر زیر ایجاد شوند:

E:\Afe\Projects\CoreCMS\Velora.Application.Shared\Resources\FormResources

نام‌گذاری فایل‌ها 

برای هر Entity باید دقیقاً دو فایل Resource ایجاد شود:

فایل انگلیسی:

EntityName>.en.resx>

فایل فارسی:

EntityName>.fa.resx>

مثال:

اگر Entity برابر باشد با ComponentType:

ComponentType.en.resx

ComponentType.fa.resx

📌 نحوه تعریف عنوان فیلدها

در داخل فایل‌های resx باید برای هر پراپرتی، عنوان مناسب همان زبان تعریف شود.

ساختار کلیدها:

<EntityName>.<PropertyName>

📌 مثال:

برای ComponentType:

GB فایل ComponentType.en.resx

ComponentType.Code = Code

ComponentType.Name = Name

ComponentType.Description = Description

IR فایل ComponentType.fa.resx

کد = ComponentType.Code

نام = ComponentType.Name

توضیحات = ComponentType.Description

مرحله ۵: اضافه کردن منو و صفحه فرم جدید در SeedData.json

برای هر فرم جدید، باید منو و صفحه مربوطه را به فایل SeedData.json اضافه کنید تا در محیط توسعه و پروژه بارگذاری شود.

📌 مسیر فایل

E:\Afe\Projects\CoreCMS\Velora.Application.Shared\SeedData.json

📌 ساختار منو و صفحه

نوع منو:

"Type": "MENU" → منو

"Type": "PAGE" → صفحه مرتبط با فرم

فیلدهای کلیدی:

فیلد توضیح

Code کد یکتا برای منو یا صفحه

Name نام انگلیسی

DisplayName نمایش متن در UI، شامل en و fa

Order ترتیب نمایش منو یا صفحه

Roles نقش‌هایی که دسترسی دارند (اختیاری، معمولاً ["DEV"])

Children آیتم‌های فرزند (زیرمنو یا صفحه)

🔴 مثال اضافه کردن فرم جدید (ComponentType)

این مثال مشابه فرم ComponentType است که به منوی اطلاعات پایه اضافه شده است:

```
}  
  , "Type": "MENU"  
  , "Code": "COMPONENTTYPE_MANAGEMENT"  
  , "Name": "ComponentType Management"  
  , "DisplayName": "  
    , "en": "ComponentType Management"  
    , "fa": "مدیریت نوع کامپوننت"  
    , {  
      , "Order": 3  
      , "Roles": [ "DEV" ]  
      , "Children":  
    }  
  }
```

```

        , "Type": "PAGE"
    , "Code": "COMPONENTTYPE_MANAGEMENT_PAGE"
    , "Name": "ComponentType Management Page"
    } : "DisplayName"
    , "en": "ComponentType Management Page"
    "fa": "صفحه مدیریت نوع کامپوننت"
    , {
        "Order": 1
    }
    [
    {

```

🔴 نکات کلیدی

کدها یکتا باشند:

Code برای منو و صفحه نباید با سایر منوها تداخل داشته باشد.

ترتیب نمایش:

Order مشخص می کند ترتیب نمایش منو و صفحه در UI است.

ارتباط با منوی پدر:

برای اتصال فرم جدید به منوی اطلاعات پایه یا سایر منوها، باید در بخش Children منوی پدر اضافه شود.

نقش ها (Roles):

معمولاً فرم ها و منوها برای "DEV" در محیط توسعه هستند، اما می توانید بر اساس نیاز سایر نقش ها را هم اضافه کنید.

تطابق با Model و Resource:

Code منو و صفحه باید با نام Entity و فایل Resource هماهنگ باشد تا نمایش عناوین فارسی و انگلیسی به درستی انجام شود.

✅ با اضافه کردن این بخش، فرم جدید به سیستم معرفی می شود و در محیط توسعه در منوهای UI و صفحه مرتبط قابل دسترسی خواهد بود.

مرحله ۶: تعریف Service و Interface فرم

برای هر فرم جدید باید یک Service اختصاصی و Interface مربوطه تعریف شود.

۶-۱: Interface سرویس

برای هر Entity باید یک Interface ساخته شود:

IComponentTypeService

این Interface مسئول تعریف عملیات‌های اصلی فرم است.

۶-۲: کلاس Service

برای هر فرم یک Service مطابق الگوی زیر ایجاد می‌شود:

```
public class ComponentTypeService
    GenericService<SqlComponentType, SqlComponentType, ComponentTypeDto>, :
        IComponentTypeService
```

نکات مهم معماری Service 

۱ ارث‌بری (Inheritance)

همه سرویس‌ها باید از GenericService ارث ببرند.

ورودی‌ها معمولاً شامل:

Entity SQL

View SQL

DTO

۲ مسئولیت Service

این سرویس مسئول موارد زیر است:

Create ✓ (ایجاد رکورد)

Update ✓ (ویرایش رکورد)

Delete ✓ (در صورت وجود)

Excel Bulk Insert ✓

Excel به Export ✓

Read/List → در GraphQL انجام می‌شود (خیلی مهم) ✗

۶-۳: نکته خیلی مهم (Architecture Rule) ✦

🔔 عملیات Get/List (خواندن لیست داده‌ها) داخل Service انجام نمی‌شود

و در مرحله بعد توسط GraphQL مدیریت خواهد شد.

۶-۴: نمونه متدهای اصلی ✦

Create ✓

```
public async Task<ResultDto<ComponentTypeDto>> CreateAsync(ComponentTypeCrud input)
```

تبدیل Crud → DTO

ثبت داده

Commit Transaction

Update ✓

```
public async Task<ResultDto<ComponentTypeDto>> UpdateAsync(ComponentTypeCrud input)
```

بررسی وجود Id

Mapping به DTO

Update

Commit Transaction

Bulk Insert (Excel) ✓

```
public async Task<ResultDto<BulkInsertResult>> BulkInsertAsync(Stream excelStream)
```

ویژگی‌ها:

خواندن Excel

تبدیل به Crud Model

ثبت هر رکورد جداگانه

جمع‌آوری خطاها

تولید فایل خطا در صورت وجود مشکل

Export Excel ✓

```
public async Task<byte[]> ExportAsync(bool exportCurrentPage, int pageNumber, int pageSize)
```

ویژگی‌ها:

گرفتن داده‌ها از View

Paging (در صورت نیاز)

تولید Excel Template

Fill کردن داده‌ها داخل Template

📌 ۵-۶: متد GetAllViews

```
(public async Task<IQueryable<ComponentTypeCrud>> GetAllViews  
}
```

```
return await GetAllViewQueryable<SqlComponentTypeView, SqlComponentTypeView,  
;()<ComponentTypeCrud  
{
```

این متد فقط برای Export و عملیات داخلی استفاده می‌شود

جایگزین GraphQL نیست

Constructor Service : ۶-۶

در Constructor باید موارد زیر Inject شوند:

Repository ها (SQL / PostgreSQL)

AutoMapper

TransactionService

Localization Service

ExcelService

CurrentUserService

Configuration

Environment

۶-۷: نکات مهم (خیلی مهم)

همه عملیات‌های Create/Update باید داخل Transaction باشند

در صورت Exception باید Rollback انجام شود

پیام‌ها باید از LocalizationMessageService گرفته شوند

Mapping بین DTO → Crud دستی انجام می‌شود

هیچ Logic مربوط به UI نباید داخل Service باشد

Read/List فقط از طریق GraphQL Layer انجام می‌شود

جمع‌بندی مرحله ۶

✓ Service فقط مسئول Business Logic است

✓ Service CRUD + Bulk + Export داخل

✓ Read/List فقط GraphQL

✓ Interface همیشه همراه Service ساخته می‌شود

✓ GenericService پایه تمام سرویس‌هاست

مرحله ۷: پیاده‌سازی GraphQL برای خواندن داده‌ها (Query Layer)

در این مرحله، عملیات خواندن لیست داده‌ها (Read/List) از طریق GraphQL انجام می‌شود.

🔔 نکته بسیار مهم:

در این معماری، هیچ Read یا List داخل Service انجام نمی‌شود و فقط از GraphQL انجام می‌گردد.

🔖 ۷-۱: ساخت Resolver (GraphQL Query)

برای هر Entity باید یک کلاس Resolver ساخته شود.

👤 قانون نام‌گذاری:

کلاس‌ها و Interface ها باید حتماً به GqlResolver ختم شوند:

ComponentTypeGqlResolver

IComponentTypeGqlResolver

🔖 نمونه کامل Resolver:

[ExtendObjectType("Query")]

```
public class ComponentTypeGqlResolver : IComponentTypeGqlResolver
{
```

```
    ;private readonly IComponentTypeService _componentTypeService
```

```
    public ComponentTypeGqlResolver(IComponentTypeService componentTypeService)
    {
```

```
        ;componentTypeService = componentTypeService_
```

```

{

    [Authorize]
    [GraphQLName("componentTypeView")]
    [UsePaging(IncludeTotalCount = true)]
    [UseFiltering]
    [UseSorting]

    ()public async Task<IQueryable<ComponentTypeCrud>> componentTypeView

    }

    var query = await _componentTypeService
    GetAllViewQueryable<SqlComponentTypeView, SqlComponentTypeView, .
        ;()<ComponentTypeCrud

    return query.Select(x => new ComponentTypeCrud

        }

        ,Id = x.Id
        ,Code = x.Code
        ,Name = x.Name
        ,Type = x.Type
        ,Description = x.Description

        // 🚨 جلوگیری از null (خیلی مهم)
        ,IsActive = x.IsActive ?? false
        ,"" ?? CreatedAtPersian = x.CreatedAtPersian
        ,"" ?? UpdatedAtPersian = x.UpdatedAtPersian
        ,"" ?? CreatedByName = x.CreatedByName
        ,"" ?? UpdatedByName = x.UpdatedByName

```

ShouldInsert = x.ShouldInsert

```
;({  
  {  
    {
```

📌 ۷-۲: نکات بسیار مهم (Critical Rules)

❏ جلوگیری از Null (اجباری)

تمام فیلدهایی که امکان null دارند باید حتماً اصلاح شوند:

,IsActive = x.IsActive ?? false

, "" ?? CreatedByName = x.CreatedByName

, "" ?? UpdatedAtPersian = x.UpdatedAtPersian

🚨 اگر null کنترل نشود، GraphQL یا UI خطا خواهد داد.

❏ نام متد GraphQL (خیلی مهم)

نام متد باید دقیقاً به این شکل باشد:

EntityName> + View (camelCase)>

مثال:

componentTypeView

roleView

resourceView

❏ View مورد استفاده

حتماً باید از View مخصوص SQL استفاده شود:

SqlConnectionTypeView

۷-۳: Global Using ها (اجباری) 

تمام Entity ها باید در globalUsing.cs تعریف شوند:

```
global using SqlConnectionType =  
;Velora.EntityFrameworkCore.EntityFramework.SqlServer.ComponentType  
  
global using PgComponentType =  
;Velora.EntityFrameworkCore.EntityFramework.SqlServer.ComponentType  
  
global using SqlConnectionTypeView =  
;Velora.EntityFrameworkCore.EntityFramework.SqlServer.VwComponentTypeForm
```

 بدون این تعریف‌ها، GraphQL Resolver کامپایل نمی‌شود.

۷-۴: ثبت GraphQL در Program.cs 

در تنظیمات GraphQL باید Resolver اضافه شود:

```
var gqlBuilder = builder.Services  
    .AddGraphQLServer.  
    .AddAuthorization.  
    .AddQueryType.  
    .AddFiltering.  
    .AddSorting();
```

 نکته مهم (مشکل Context Conflict)

اگر از AddTypeExtension() استفاده شد:

AddTypeExtension<ComponentTypeGqlResolver>()

باید حتماً بررسی شود:

یا از ExtendObjectType استفاده کنید

یا AddTypeExtension

✗ هر دو همزمان باعث خطای:

context is already in use

📌 ۷-۵: خلاصه قوانین GraphQL

✓ تمام Read/List ها فقط اینجا انجام می شود

✓ Service هیچ Get/List ندارد

✓ Resolver باید به GqlResolver ختم شود

✓ Null ها حتماً کنترل شوند

✓ نام Query = entity + View (camelCase)

✓ View SQL باید از global using بیاید

✓ Filtering + Sorting + Paging اجباری است

🎯 جمع بندی مرحله ۷

این مرحله تعیین می کند که:

داده ها چگونه از سیستم خوانده شوند

Pagination چگونه انجام شود

Sorting و Filtering چگونه فعال شوند

UI چگونه داده‌ها را دریافت کند

📌 ۸-۱ Mapping های الزامی

برای هر فرم جدید باید این ۳ Mapping حتماً اضافه شود:

Entity ↔ CRUD ۱

```
;()CreateMap<SqlComponentType, ComponentTypeCrud>().ReverseMap
```

✓ استفاده در:

فرم‌ها (Create / Update)

DataGrid

Bulk Insert

View ↔ CRUD ۲

```
;()CreateMap<SqlComponentTypeView, ComponentTypeCrud>().ReverseMap
```

✓ استفاده در:

GraphQL Query

نمایش لیست‌ها

Filtering / Sorting / Paging

Entity ↔ DTO ۳

```
;()CreateMap<SqlComponentType, ComponentTypeDto>().ReverseMap
```

✓ استفاده در:

Service Layer

Create / Update عملیات‌ها

انتقال داده بین لایه‌ها

📌 ۸-۲: نکات بسیار مهم (Critical Rules)

🔔 ۱. نبود Mapping باعث خطای Runtime می‌شود

اگر حتی یکی از Mapping ها تعریف نشود:

داده‌ها Null می‌شوند

یا AutoMapper Exception می‌دهد

🔔 ۲. View و Entity هر دو باید Map شوند

حتماً باید هر دو مسیر پوشش داده شوند:

Entity → CRUD

View → CRUD

🔔 ۳. ReverseMap اجباری است

تمام Mapping ها باید شامل ReverseMap () باشند:

.ReverseMap();

✓ چون سیستم در هر دو جهت (خواندن و نوشتن) از آن استفاده می‌کند.

📌 ۸-۳: الگوی استاندارد برای هر Entity

برای هر فرم جدید همیشه این ۳ خط باید اضافه شود:

```
;() CreateMap<Sql<Entity>, <Entity>Crud>().ReverseMap
```

```
;() CreateMap<Sql<EntityView>, <Entity>Crud>().ReverseMap
```

```
;() CreateMap<Sql<Entity>, <Entity>Dto>().ReverseMap
```

🎯 مثال واقعی (ComponentType)

```
;() CreateMap<SqlComponentType, ComponentTypeCrud>().ReverseMap
```

```
;() CreateMap<SqlComponentTypeView, ComponentTypeCrud>().ReverseMap
```

```
;() CreateMap<SqlComponentType, ComponentTypeDto>().ReverseMap
```

🔴 ۴-۸: خلاصه قوانین مرحله ۸

✓ همه Mapping ها داخل MappingProfile باشند

✓ Entity ↔ CRUD حتماً تعریف شود

✓ View ↔ CRUD حتماً تعریف شود

✓ Entity ↔ DTO حتماً تعریف شود

✓ ReverseMap اجباری است

✓ بدون این مرحله سیستم GraphQL + Service + Form خراب می شود

9. Entity در ModelMapping ثبت

ثبت شود ModelMapping داخل Entity کند، حتماً باید Cast و Resolve بتواند مدل را Export Excel برای اینکه

نمونه:

```
public static class ModelMapping
{
    private static readonly Dictionary<string, Type> _map =
        new (StringComparer.OrdinalIgnoreCase)
        {
            { LookupEntities.Resource, typeof(ResourceCrud) },
            { LookupEntities.User, typeof(UserCrud) },
            { LookupEntities.ResourceType, typeof(ResourceTypeCrud) },
            { LookupEntities.Role, typeof(RoleCrud) },
        }
```

```

        { LookupEntities.ComponentType, typeof(ComponentTypeCrud) }
    };

    public static Type? GetModelType(string entityName)
    {
        return _map.TryGetValue(entityName, out var type)
            ? type
            : null;
    }
}

```

ثبت نشود ModelMapping داخل Entity در صورتی که:

- پیدا نمی‌شود Export مدل هنگام
- انجام نمی‌شود Cast عملیات
- با خطا مواجه می‌شود Excel تولید فایل
- خواهد شد fail در فرانت یا بک‌اند Dynamic Export



منظورم کفش بود شماره موردش بشه ۹

9. Export و BulkInsert برای Dynamic الزامات فرم‌های

را Export و BulkInsert در سیستم ایجاد می‌شود، الزاماً باید متدهای Dynamic که برای فرم‌های Controller هر پیاده‌سازی کند

در غیر این صورت:

- ایجاد نمی‌شوند Export و BulkInsert های مربوط به عملیات Resource
- ثبت نمی‌شوند Developer ها برای نقش Permission
- مواجه خواهد شد Service not found فرانت‌اند هنگام فراخوانی سرویس‌ها با خطای
- یا ثبت دسته‌ای اطلاعات کار نخواهد کرد Excel دانلود فایل

9.1 BulkInsert متد

ها باید متد زیر را داشته باشند Controller تمام

```

[HttpPost("BulkInsert")]
[Consumes("multipart/form-data")]
[RequestFormLimits(MultipartBodyLengthLimit = 50_000_000)] // 50 MB
public async Task<IActionResult> BulkInsert()
{
    if (!Request.HasFormContentType)
        return BadRequest(new { Message = "Invalid content type, expected multipart/form-data." });

    var form = await Request.ReadFormAsync();
    var file = form.Files.FirstOrDefault();

    if (file == null || file.Length == 0)

```

```

        return BadRequest(new { Message = "File is required." });

using var stream = file.OpenReadStream();

var result = await _service.BulkInsertAsync(stream);

var bulkResult = new BulkInsertResult
{
    InsertedCount = result.Data?.InsertedCount ?? 0,
    ErrorCount = result.Data?.ErrorCount ?? 0,
    ErrorFileUrl = result.Data?.ErrorFileUrl
};

await _transactionService.CommitAsync();

return Ok(new ResultDto<BulkInsertResult>
{
    Success = result.Success,
    Message = result.Message,
    Data = bulkResult,
    Errors = result.Errors
});
}

```

9.2 متد Export

ها بايد متد زير را داشته باشند **Controller** تمام:

```

[HttpPost("Export")]
[AllowAnonymous]
public async Task<IActionResult> Export([FromBody] ExportRequestDto request)
{
    if (request == null)
        return BadRequest(new { Success = false, Message = "Invalid request"
});

    byte[] fileBytes;

    try
    {
        fileBytes = await _service.ExportAsync(
            request.ExportCurrentPage,
            request.PageNumber,
            request.PageSize
        );
    }
    catch (Exception ex)
    {
        return StatusCode(500, new
        {
            Success = false,
            Message = "Error exporting data",
            Details = ex.Message
        });
    }
}

```

```
}

var fileName = $"{nameof(Entity)}_{DateTime.Now:yyyyMMdd_HH:mm:ss}.xlsx";

return File(
    fileBytes,
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    fileName
);
}
```

9.3 پاک کردن SeedHistory

Export/BulkInsert جدید یا متدهای Controller بعد از اضافه شدن

پاک شود SeedHistory باید اطلاعات جدول

نمونه:

```
DELETE FROM SeedHistory
```

سپس پروژه مجدداً اجرا شود.

9.4 دلیل پاک کردن SeedHistory

Seeder با اجرای مجدد

- های مربوط به Resource:
 - Export
 - BulkInsert

ثبت می شوند.

همچنین:

- Permission ها ایجاد می شوند
 - RolePermission ثبت می شود Developer برای نقش
 - Dynamic Resolve فرانت اند می تواند سرویس ها را کند
-

9.5 Entity ثبت در ModelMapping

ثبت شود ModelMapping داخل Entity کند، حتماً باید Cast و Resolve بتواند مدل را Export Excel برای اینکه

نمونه:

```
public static class ModelMapping
{
    private static readonly Dictionary<string, Type> _map =
        new(StringComparer.OrdinalIgnoreCase)
        {
            { LookupEntities.Resource, typeof(ResourceCrud) },
            { LookupEntities.User, typeof(UserCrud) },
            { LookupEntities.ResourceType, typeof(ResourceTypeCrud) },
            { LookupEntities.Role, typeof(RoleCrud) },
            { LookupEntities.ComponentType, typeof(ComponentTypeCrud) }
        };

    public static Type? GetModelType(string entityName)
    {
        return _map.TryGetValue(entityName, out var type)
            ? type
            : null;
    }
}
```

ثبت نشود ModelMapping داخل Entity در صورتی که:

- پیدا نمی‌شود Export مدل هنگام
- انجام نمی‌شود Cast عملیات
- با خطا مواجه می‌شود Excel تولید فایل
- خواهد شد fail در فرانت یا بک‌اند Dynamic Export

مرحله ۱۰: تعریف مدل TypeScript و Entity Name در Frontend

در این مرحله باید مدل TypeScript مربوط به فرم و همچنین نام Entity ها در فرانت‌اند تعریف شود.

📌 ۱۰-۱: تعریف مدل TypeScript

برای هر فرم یک **interface** در مسیر زیر ساخته می‌شود:

E:\Afe\Projects\Velora-Ui\apps\admin\src\types\models

نام گذاری:

EntityName>Crud>

مثال:

```
} export interface ComponentTypeCrud
```

```
;id?: string
```

```
;Name?: string
```

```
;Code?: string
```

```
;Type?: string
```

```
;Description?: string
```

```
;CreatedAtPersian?: string
```

```
;UpdatedAtPersian?: string
```

```
;CreatedByName?: string
```

```
;UpdatedByName?: string
```

```
;shouldInsert?: boolean
```

```
{
```

نکات مهم

نام فیلدها باید دقیقاً مطابق خروجی GraphQL باشد

فیلدهای سیستمی (Created/Updated) همیشه باید وجود داشته باشند

مدل‌ها فقط برای تایپ‌سیف بودن UI استفاده می‌شوند

۱-۲: ثبت Entity Name ها (خیلی مهم)

برای مدیریت استاندارد نام Entity ها باید در فایل زیر ثبت شوند:

src/constants/entityNames.ts

مثال: 

```
} = export const entityNames
  , "Role: "Role
  , "User: "User
  , "Resource: "Resource
  , "ResourceType: "ResourceType
  , "Permission: "Permission
};
```

نکات مهم Entity Names 

هر Entity جدید باید اینجا اضافه شود
برای استفاده در GraphQL / Service / UI routing استفاده می شود
باعث جلوگیری از hardcode شدن نام ها در پروژه می شود

 جمع بندی مرحله ۱۰

✓ تعریف Interface برای هر فرم الزامی است

✓ مسیر ثابت: types/models

✓ نام گذاری: <Crud> EntityName

✓ ثبت Entity جدید در entityNames.ts اجباری است

✓ هماهنگی کامل با GraphQL ضروری است

مرحله ۱۱: ایجاد Service در Frontend برای ارتباط با API

در این مرحله برای هر فرم باید یک فایل Service در فرانت اند ساخته شود تا عملیات های CRUD با Backend انجام شود.

هدف این مرحله 

این سرویس‌ها مسئول ارتباط مستقیم با API هستند:

ایجاد داده (Create)

ویرایش داده (Update)

حذف داده (Delete)

مسیر فایل‌ها 

معمولاً سرویس‌ها در مسیر زیر قرار می‌گیرند:

src/services

یا در ساختار مشابه پروژه فعلی.

الگوی نام‌گذاری 

EntityName>Service.ts>

مثال واقعی (ComponentType Service) 

```
;"import { ResultDto } from "@types/models/ResultDto";  
;"import api from "../apiClient";  
;"import { handleApiError } from "@utils/handleApiError";  
;"import { ComponentTypeCrud } from "@types/models/ComponentTypeCrud";
```

Create : ۱ - ۱ 

```
) export const createComponentType = async  
  ,data: ComponentTypeCrud  
{ <= <<Promise<ResultDto<ComponentTypeCrud | null :(  
  } try  
<<const res = await api.post<ResultDto<ComponentTypeCrud  
  ,"api/ComponentType/Create/"
```

```

        ,data
        { withCredentials: true }
        ;(
        ;return res.data
        } (catch (error: unknown {
;return handleError<ComponentTypeCrud>(error)
        {
        ;{
Update : \ . - ۲ 📌
    ) export const updateComponentType = async
        ,data: ComponentTypeCrud
    } <= <<Promise<ResultDto<ComponentTypeCrud | null :{
        } try
    )<<const res = await api.put<ResultDto<ComponentTypeCrud
        ,"api/ComponentType/Update/"
        ,data
        { withCredentials: true }
        ;(
        ;return res.data
        } (catch (error: unknown {
;return handleError<ComponentTypeCrud>(error)
        {
        ;{
Delete : \ . - ۳ 📌
    ) export const deleteComponentType = async
        ,id: string | number
    } <= <<Promise<ResultDto<ComponentTypeCrud | null :{

```

```

    } try
) <<const res = await api.delete<ResultDto<ComponentTypeCrud
    , `api/ComponentType/${id}`
    { withCredentials: true }
    ;(
    ;return res.data
    } (catch (error: unknown {
;return handleApiError<ComponentTypeCrud>(error)
    {
    ;{

```

نکات مهم (Important Rules) 

۱ نام API باید استاندارد باشد

api/<EntityName>/Create/

api/<EntityName>/Update/

api/<EntityName>/{id}/

۲ همه درخواست‌ها باید با ResultDto باشند

تمام خروجی‌ها باید از ساختار زیر پیروی کنند:

<ResultDto<T

۳ مدیریت خطا اجباری است

تمام متدها باید از این استفاده کنند:

handleApiError

۴ withCredentials الزامی است

برای احراز هویت:

```
{ withCredentials: true }
```

الگوی استاندارد برای هر Entity 

```
... = <export const create<EntityName
```

```
... = <export const update<EntityName
```

```
... = <export const delete<EntityName
```

جمع‌بندی مرحله ۱۱ 

✓ هر Entity باید یک Service در Frontend داشته باشد

✓ مسیر ثابت: src/services

✓ عملیات‌ها: Create / Update / Delete

✓ همه API ها باید از ResultDto استفاده کنند

✓ خطاها باید با handleError مدیریت شوند

✓ withCredentials اجباری است

مرحله ۱۲: ساخت صفحات در Next.js و تنظیم Route

در این مرحله باید صفحات مربوط به هر فرم در پروژه Frontend (Next.js) ساخته شوند و داخل مسیر درست منوی قرار بگیرند.

مسیر اصلی صفحات 

تمام صفحات ادمین در مسیر زیر قرار می‌گیرند:

E:\Afe\Projects\Velora-Ui\apps\admin\src\app\admin

ساختار فولدرها (خیلی مهم) 

هر فرم باید داخل فولدر مربوط به منوی خودش قرار بگیرد:

مثال ساختار:

```

/admin
  /basic-info —|
    /componenttype-management —| |
      page.tsx —| | |
        ComponentTypeFormPage.tsx —| | |
          ComponentTypeReadOnlyPage.tsx —| | |
```

🔴 ۱۲-۱: فایل‌های مورد نیاز هر فرم

برای هر Entity باید این ۳ فایل ایجاد شود:

page.tsx ❏ ۱

صفحه اصلی Route در Next.js

Entry point برای آن فرم

FormPage.tsx ❏ ۲

صفحه ایجاد و ویرایش (CRUD Form)

ReadOnlyPage.tsx ❏ ۳

صفحه نمایش فقط خواندنی (View Mode)

🔴 ۱۲-۲: قوانین نام گذاری

🔴 نام فولدر:

<EntityName>-management

🔴 مثال:

componenttype-management

permission-management

role-management

📌 ۱۲-۳: ساختار Route (خیلی مهم)

Route باید دقیقاً مطابق منو در سیستم باشد:

مثال:

basic-info/componenttype-management

یا برای بخش دیگر:

administration/user-management

📌 ۱۲-۴: بررسی رکورد منو و Route (اصلاح شد 🔥)

حتماً باید رکورد منو و صفحه مربوطه برای این فرم در جدول Resources بررسی شود

فیلد Route نباید null باشد

Route باید دقیقاً با مسیر فولدر Next.js و SeedData.json مطابقت داشته باشد

مثال:

basic-info/componenttype-management

اگر Route null باشد یا اشتباه باشد، صفحه نمایش داده نمی‌شود و Navigation خراب می‌شود.

📌 ۱۲-۵: ارتباط با منوها

هر صفحه باید دقیقاً زیر منوی خودش قرار بگیرد:

مثال:

Basic Info

ComponentType Management — L

✓ هر فرم = ۳ صفحه (page / form / readonly)

✓ مسیرها باید مطابق ساختار منو باشند

✓ فولدرها در admin/app ساخته می‌شوند

✓ رکورد منو و صفحه در جدول Resources باید بررسی و Route نباید null باشد

✓ Next.js App Router مسئول routing نهایی است